

AgentWorkShop: A Node-Native Integration Framework for LLM Multi-Agent Governed Closed-Loop Optimization of Heterogeneous Industrial Production Lines

First Author, Second Author, and Third Author

Abstract—Industrial LLM-agent deployments stop at the advisory boundary: published systems diagnose, generate PLC code, or recommend setpoints; none integrates an agent team into a production line’s own protocol, data, and semantic fabric. We present AgentWorkShop, a configuration-driven integration framework that embeds LLM multi-agent teams natively into the supervisory fabric of heterogeneous production lines: a multi-harness agent runtime (14 interchangeable engines) and an industrial runtime (five protocol drivers with per-node edge acquisition) execute as one process over one data domain. Commissioning is configuration, not code: one plant-scenario identifier re-instantiates the deployment on a second, independently defined line with zero platform code changes; across both scenarios, all 29 seeded out-of-window/range writes are intercepted with zero false blocks. Every agent-issued write traverses one governed path—recipe-window interlock, mode-bound approval, readback-verified write, attributable journal—so the accepted-action language is constrained by construction, and the backstop’s automatic rollback is exercised end-to-end on a live breach. A governed closed-loop benchmark on a cast-film twin recovers 96.2–97.3 % of the offline grid optimum under a fixed deterministic controller (no LLM in the loop; quoted run: 97.0–97.2 %, in two iterations with at most two governed writes per seed); four uncontrolled same-seed LLM-agent-team campaigns closed the same loop over real transports with every write attributed, but reached only 0.35–0.76 (median 0.76) of that optimum—a measured shortfall that motivates the framework’s propose/dispose design. All results come from one executable pipeline with machine-checkable reproduction against frozen baselines; all measured layers are simulation-tier.

Index Terms—Agent integration framework, closed-loop optimization, digital twin, human-in-the-loop, industrial automation, large language models, multi-agent systems, node-native integration, protocol heterogeneity.

I. INTRODUCTION

INDUSTRIAL enterprises are deploying large language model (LLM) agents widely [1], [2]. Yet essentially all published industrial LLM systems stop at the *advisory boundary*: multi-agent frameworks build industrial knowledge graphs [3], generate and verify PLC code [4], [5], [6], and parameterize digital-twin simulations [7], [8]; beyond single-stack control interfaces [9] and simulated fault-tolerant control [10], in none does an agent team’s decision become a governed write on a protocol-heterogeneous plant. The boundary is well

motivated—an unguarded write path from a stochastic model to physical actuators is unacceptable in production—but it has left the central question quantitatively unanswered: *what does it cost, in integration effort and latency, to let an LLM agent team close the control loop on a real plant, and what safety does the resulting governance buy?* This paper measures both costs on a live software instance driving real protocol transports against seeded plant simulators.

Answering them requires three things no existing work provides together: (i) a *node-native integration substrate*—agents, protocol drivers, and plant data in one process and one data domain, so binding an agent to an actuator is a configuration act; (ii) a *governed write path* whose accepted-action language is safe by construction, attributable, recoverable, and formalized enough to be checked rather than asserted; and (iii) a *reproducible benchmark* measuring integration adaptability and the write path’s safety–overhead trade-off across protocols, plant scenarios, and fault conditions.

We present AGENTWORKSHOP, a configuration-driven integration framework that meets all three requirements, together with its evaluation suite, AW-INDUSTRIALBENCH. It couples an *agent half* (14 interchangeable LLM harnesses, lead-based team supervision, episodic–semantic memory) with an *industrial half* (five protocol drivers, per-node edge acquisition, a hysteresis-and-debounce alarm chain, time-series/frame storage); the halves meet at a *governed bridge*. Nodes are bound to agents, physical semantics are injected as semantic cards, and the agent tool surface exposes exactly one write path: a batch-scoped recipe-window *interlock*, a mode-bound human-approval *gate*, a readback-verified *write*, and a signed, attributable *journal*. Throughout, *closed loop* means the supervisory optimization loop—setpoint-level decisions at a seconds-to-minutes cadence; regulatory control remains in the PLCs. A system-level *backstop* scores open optimization records against the recipe window and executes bounded rollbacks. Because the agent runtime and the node drivers share one process, this contract is enforced on the platform-exposed write path rather than merely declared (the containment boundary around the harness engines themselves is a deployment property; Sec. IV-G). Commissioning is configuration, not code: driver configurations derive at runtime from each device’s integration export, and a named *plant scenario* re-instantiates the full deployment (measured in Sec. V-B).

Our contributions are:

Manuscript received XXXX; revised XXXX. (Corresponding author: First Author.)

The authors are with XXXX, City, Country (e-mail: first.author@xxxx.edu).

- 1) **A node-native integration framework** for LLM multi-agent teams on heterogeneous production lines: agent and industrial runtimes in one process and one data domain; five protocol drivers with per-node edge acquisition; physical-semantic grounding via semantic cards; team-level node binding under a 14-engine harness registry (Sec. III).
- 2) **A governed write-control pipeline** for agent-issued actuator writes, formalized as a supervised state machine with three safety invariants—containment, readback closure, bounded autonomy—each mapped to implementation anchors and negative acceptance tests (Secs. IV-C–IV-G).
- 3) **AW-INDUSTRIALBENCH, an executable benchmark protocol**: seeded, machine-checkable evaluation spanning a static artifact audit, live-instance invariant verification, five-protocol real scenarios, cross-scenario portability, a closed-loop optimization benchmark, and a four-arm governance ablation; frozen baselines and a comparison gate turn reproduction into a machine verdict (Sec. V). The measured ablation shows the cost is structural, not temporal (Sec. V-G).
- 4) **A completed closed-loop program on a seeded extrusion cast-film line**: a fixed deterministic controller (no LLM in the loop) recovers $J/J^* = 97.0\text{--}97.2\%$ of the offline grid optimum ($W^* = 89.894$) in two iterations and at most two governed writes per plant seed, and four uncontrolled same-seed LLM-agent-team campaigns close the same loop over real transports with every write attributed but reach only 0.35–0.76 (median 0.76) of that optimum—a measured negative that motivates the propose/dispose split (Secs. V-F–V-H).

II. RELATED WORK

A. Advisory-Boundary Systems

Industrial LLM applications cluster into advisory families. *Diagnosis and assistance*: CoMA-IKG builds industrial knowledge graphs with LLM agents [3]; joint KG–LLM pipelines diagnose aviation-assembly faults [11]; surveys chart the breadth [1], [12], [13]. *Engineering automation* generates control code: Agents4PLC verifies PLC code against formal specifications [4], as does LLM4PLC via compilers and model checkers [5]; a multi-task LLM system drafts PLC programs [14]; MetaIndux-PLC adds control-logic guidance [6]; embodied multi-agent systems couple LLMs to physical stations and shopfloor scheduling [15], [16], and agent-based supervisors coordinate service-oriented industrial cyber-physical systems [17]. *DT-coupled agents* parameterize simulations [7], survey LLM-enhanced twin modeling [18], and use a twin as prompt source and validation sandbox [8]. All share two boundaries: output reaches a human or a compiler, never an actuator; and the DT twin is the *destination* of agent action, whereas in AGENTWORKSHOP it merely observes a real protocol stack—the destination is the plant. *Closed-loop LLM control*, the newest family, removes the boundary itself: LLM agents have turned fault detections into fault-tolerant control actions on simulated plants [10], and tool-equipped LLMs have controlled real machines over OPC

UA in natural language [9]; in InstructMPC, human instructions reach a model-predictive controller’s *predictor* through an LLM in closed loop [19]. Adjacent-domain actuation—autonomous chemistry [20], robot control [21], closed-loop code generation for robotic calibration [22]—targets single-apparatus loops, not production lines. These works couple one agent (or team) to one protocol stack with per-application governance; none contributes a reusable integration substrate (protocol-heterogeneous, node-native, team-level binding) or a reproducible governed-write benchmark—the gap AGENTWORKSHOP fills. The alarm chain of Sec. IV-D follows alarm-management practice (ISA-18.2) [23].

B. Governed Execution, Benchmarks, and Frameworks

Outside industry, τ -bench [24] supplies our approval-gate evaluation paradigm (as does the consequential-task benchmark TheAgentCompany [25]). Hard action enforcement—control barrier functions [26], shielding [27]—has its own line; here the wrapped component is an *agent tool surface*, whose fallback path (interlock, approval, readback, journal) is part of the audited contract. Approval semantics rest on levels-of-automation research [28]. The line/product/recipe/run tree of Sec. IV-D combines ISA-95 [29] with the ISA-88 recipe/batch model [30]; the semantic card is the node-level counterpart of the Asset Administration Shell (IEC 63278) [31]. General agent benchmarks [32] involve no physical plant or actuator safety contract; industrial datasets (TEP [33], SWaT [34]) target fault diagnosis, not closed-loop control—AW-INDUSTRIALBENCH bridges the two. Orchestration frameworks (AutoGen [35], LangGraph [36], CrewAI [37]; on ReAct [38]) provide scheduling but no protocol layer, semantic grounding, or write governance; AW-INDUSTRIALBENCH defines them as baselines over the same MCP tool surface [39] in its released protocol. Network segmentation follows IEC 62443 [40], complementing IEC 61511’s basic-control/safety separation [41]; AGENTWORKSHOP applies it at the agent–plant layer.

III. SYSTEM ARCHITECTURE

AGENTWORKSHOP (v0.7.39) is an integration framework, not an application. In one Node.js process it ships the two halves that industrial deployments otherwise bolt together by hand—a REST and WebSocket API, an MCP server with 25 tools [39], an A2A agent interface [42], and a hot-reloading, error-isolated plugin host—under one authentication layer. Operators enter through a web console or a TUI workbench; agents enter through MCP or A2A; both meet on the same objects and the same approval queue. Fig. 1 shows the architecture; the rest of this section explains what holds it together, and Sec. IV details each mechanism.

A. Design Principles

PI, node-native integration: the agent and industrial runtimes execute in one process and one data domain, so binding, telemetry, and write control share invariants rather than APIs. This yields *check-then-actuate integrity*: the interlock test and

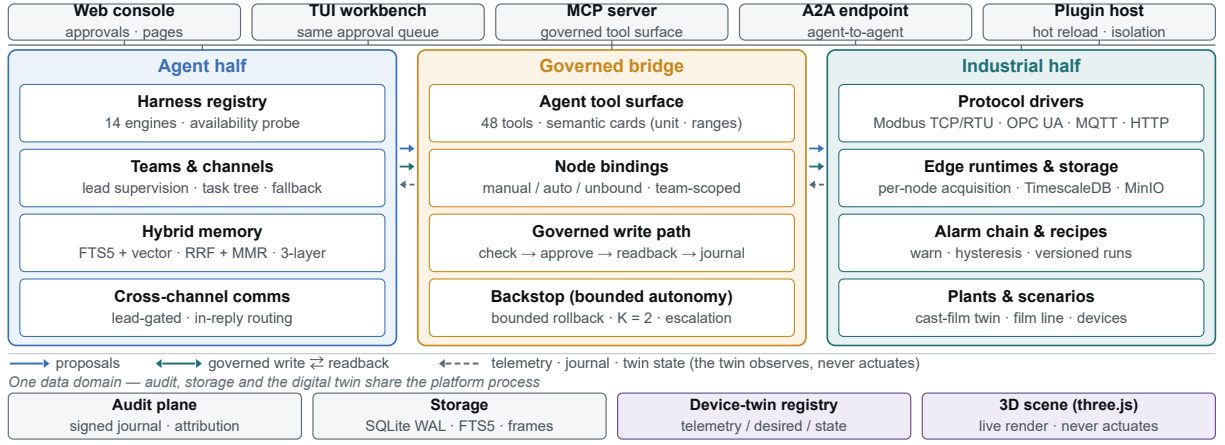


Fig. 1. AGENTWORKSHOP integration framework. **Left (agent half):** 14-engine harness registry, lead-supervised teams, episodic–semantic memory. **Center (governed bridge, amber):** the only platform-exposed path from agent intent to actuation—semantic cards, node bindings, Algorithm 1’s governed write path, the bounded-autonomy backstop. **Right (industrial half):** five protocol drivers, edge runtimes, starvation-free gateway sweep, alarm chain, line/recipe management; connectivity ends at real devices or the two seeded plant scenarios. **Top (entry surfaces):** web console, TUI workbench, MCP server, A2A endpoint, plugin host—each dropping into the same platform process. **Bottom (one data domain):** audit journal, storage, device-twin registry feeding the three.js scene; the twin renders live state but never actuates.

the driver call run in one event-loop turn over the same process-local recipe-run state, and the window that authorizes a write cannot change between check and actuation. Co-residency governs the platform-internal path but does not contain the harness engines, which remain privileged local processes (containment is a deployment property; Sec. IV-G). *P2, supervisory positioning:* AGENTWORKSHOP occupies the supervisory, soft-real-time tier; hard real-time control and safety functions remain in PLCs, mirroring the IEC 61511 separation of basic control from safety instrumented functions [41]. *P3, configuration-driven, down to the plant scenario:* a typed configuration schema drives acquisition intervals, approval policies, retention, and memory budgets; driver configurations derive from each device’s integration export, and a named *plant scenario*—devices, signal semantics, dynamics, recipes—re-instantiates the full deployment, so adapting to a new line is a configuration task (measured in Sec. V-B). *P4, engine-agnostic governance:* agent intelligence is a replaceable component; the governed write path of Sec. IV-C is defined at the tool boundary and is therefore identical for every engine.

B. Three Planes and a Governed Bridge

The architecture has three planes. The *agent half* couples 14 interchangeable LLM harness engines with lead-supervised teams (channels) and typed episodic–semantic memory, so team organization, supervision, and learning are platform services rather than per-application code (Sec. IV-B). The *industrial half* terminates five protocol families in per-node edge acquisition runtimes, a starvation-free gateway sweep, an alarm chain, and line/recipe management—the supervisory data and control substrate (Secs. IV-A and IV-D). The *governed bridge* between them is this paper’s object: binding a node to an agent injects physical semantics as a semantic card and derives a tool surface whose only write path is the governed pipeline of Sec. IV-C—48 host tools, 23 of them industrial or automated-ML, with the MCP server exposing a

25-tool read-oriented subset for external clients. Two separations hold across the whole system: *judgment is not execution*, and *lead-only tools* are restricted to lead agents with approval (Sec. IV-G).

Beneath all three planes lies *one data domain*: time series, frames, platform state, and a signed audit journal with typed actor attribution, all carrying line/product/recipe dimensions (Sec. IV-D). The device-twin registry lives here too, feeding a three.js scene that renders live state over a WebSocket bus but never actuates (Sec. IV-D).

C. One Loop, End to End

A single walk makes the collaboration concrete; Sec. IV revisits each station in order. An engineer connects a temperature controller on line 3: export its integration descriptor, test connectivity over Modbus TCP, create the node, provision line/product/recipe, and bind the node to a team (Sec. IV-A). The node now exists twice—as a physical endpoint sampled by its edge runtime, and as a *semantic card* (unit °C, hard range, recipe window, freshness) through which agents see it. A team deploys into a channel and the binding mode fixes what its writes may do unaided (Secs. IV-B and IV-G).

The plant then runs. The lead’s supervision tick notices a drift task and dispatches it; a worker agent queries fresh values (`daq_query`), reads 187 °C against a recipe window of [176, 188] °C, and proposes a setpoint move to 185 °C through `dcw_control` (Sec. IV-E). That proposal becomes a *governed write* (Sec. IV-C): the interlock checks 185 against the window, the driver writes and reads back 185, and a signed journal anchor records previous value, new value, source, and actor. Fresh acquisition confirms the controller followed; `dcw_judge` inspects the record and returns *keep*; the outcome lands in team memory (Sec. IV-B). Had the process instead drifted out of window and stayed there, the backstop would have scored the open record against the window, and after three out-of-window samples rolled the node back to

its recorded baseline—at most twice per node, then a human decides (Secs. IV-E and IV-F). Every station of this walk is configuration or platform code, none line-specific glue; Sec. V measures each of them on a live instance.

IV. CORE MECHANISMS

This section details the six mechanisms a deployment meets in order—node connection and creation (Sec. IV-A), the agent channel (Sec. IV-B), node control execution (Sec. IV-C), production-line operations (Sec. IV-D), agent closed-loop control (Sec. IV-E), and recipe management with rollback (Sec. IV-F)—and closes with the human control boundary that spans them all (Sec. IV-G). Each is the station of the Sec. III walk, seen from inside.

A. Node Connection and Creation

Five protocol drivers spanning four protocol families—Modbus TCP, Modbus RTU (over TCP), OPC UA [43], MQTT, and HTTP/REST—sit behind a uniform, plugin-registerable driver interface; a mock driver supports deterministic testing. Connecting a device is a five-step configuration flow: *export* the device’s integration descriptor, from which the driver configuration is derived; *test connectivity* against the real protocol stack; *create the node*, choosing acquisition and control roles; *provision* the line, product, and recipe objects the node joins; and *bind* the node to an agent or team. No line-specific code exists in this path, which is what makes cross-scenario recommissioning measurable (Sec. V-B).

The node carries its physical semantics as a *semantic card*: physical quantity, unit and display precision, *hard interlock* range, *soft interlock* recipe window, warn band, live value with freshness, and a rule limiting any single step to 2% of span. Agents never see register addresses; the card is their only view of the plant and the object the write path checks against (Sec. IV-C).

Each node also runs an *edge runtime* owning its sampling schedule and in-flight mutual exclusion, with three decoupled intervals: acquisition (default 5 s, floor 1 s, hard clamp [0.1, 60] s; sub-tick intervals wait for the 250 ms sweep), Web-Socket publish (default 1 s), and display/trend (default 5 s). A 250 ms acquisition-gateway sweep—an internal scheduler, not a network gateway—samples nodes under a non-accumulating budget of 64 in-flight samples per tick; a rotating cursor plus a per-node due check keeps not-yet-due nodes from consuming budget first, so tail nodes are never starved.

B. The Agent Channel: Team, Supervision, Memory

The collaboration unit is the *channel*: a named room hosting one lead, its workers, the task tree, and the message history. Teams deploy as a unit, and every industrial binding is scoped to its channel, so a lead-dispatched task and a worker’s actuation share one authorization scope. Task trees carry seven states (SUBMITTED, ASSIGNED, WORKING, WAITING for blocked parents, FAILED, CANCELLED, COMPLETED).

Each lead runs a supervision loop on a 1 s tick with exponential idle backoff and event wake-up. Each tick first assembles a state fingerprint (task tree, member queue depths, newest

mailbox message) and skips the LLM call if unchanged—an idle plant costs no tokens unless a parent task is closeable. When consulted, the LLM owns goal judgment, completion, and escalation; mechanical decisions run every tick on a deterministic rule engine: FIFO dispatch to the worker with the shortest queue (ties broken by longest idle), reassignment of failed tasks while retries remain (limit 3), and a 300 s stall watchdog treating tool activity as liveness. The rule engine doubles as the guaranteed fallback whenever the LLM abstains, errors, or returns nothing actionable.

Agent intelligence is replaceable: fourteen engines are registered with a capability matrix (mid-task steering, supervision, human-in-the-loop, among others)—an in-process deterministic mock (used in Secs. V-C–V-G), a first-party RPC engine (omp, speaking the platform’s own agent protocol), and vendor or open-source CLI/SDK agents (e.g., codex, claude, qwen). Capabilities are asserted per engine and verified by the static audit (Sec. V-C); dispatch follows an allow-list probe that resolves each executable and validates its response.

Agents accumulate typed memories (session, task, peer, team chronicle, reflection, semantic) in SQLite with FTS5 and an optional vector index. Retrieval fuses a lexical channel (FTS5, CJK-aware unigram segmentation) with a dense vector channel (sqlite-vec k -NN) by reciprocal-rank fusion [44] ($k=60$), scores candidates by $0.5\rho + 0.3\phi + 0.2\iota$ (normalized relevance, kind-aware recency with 3–14 d half-lives, access-weighted importance), diversifies by maximal-marginal-relevance [45] ($\lambda=0.7$), and injects three layers under a 500-token budget; compaction triggers at 70% of the context window and harvests summaries back into memory. Channels exchange messages only through a lead-gated cross-channel bus, and runtime extensions enter through a hot-reloading plugin host that is fault-isolated (a failing plugin cannot crash the platform) but not privilege-contained—plugins run in-process and gain no plant I/O path by construction of the tool surface.

C. Node Control Execution: The Governed Write Path

The write path is where the framework’s promise is kept: an agent proposal becomes a plant write only through one auditable sequence. A *device* is one integrated piece of plant equipment; it appears as one or more *nodes*, and n denotes a *writable* node—one exposing a setpoint, the control image of physical quantity p . n carries a hard safety range $S(n) = [n^{\min}, n^{\max}]$ and a value transform t_n ; an active *recipe run* r prescribes a process window $W_r(p)$ for p . The accepted set is the *effective constraint*

$$C(n, r) = \begin{cases} S(n) \cap W_r(p), & \text{run } r \text{ active,} \\ S(n), & \text{otherwise.} \end{cases} \quad (1)$$

Two independent mechanisms enforce containment: the interlock rejects a value inside $S(n)$ but outside $W_r(p)$, citing the window, while the driver’s engineering-quantity validation rejects out-of-range values in every state. Containment never depends on authoring-time containment of $W_r(p) \subseteq S(n)$, and both branches of Eq. (1) are exercised dynamically in Sec. V-C.

Algorithm 1 specifies the pipeline: an approval stage at the tool boundary (stage 0, Sec. IV-G), then six ordered execution stages behind the single write entry—approval is a *decision*, execution a *validated act*. (1) *Availability gate*. Unbound nodes are refused at dispatch (FORBIDDEN), before any stage runs. A paused gateway or disabled node rejects the request: governance is unavailable exactly when actuation is unavailable. (2) *Closed-loop guardrail*. Agent writes are serialized per optimization scope—one open record per agent, records idle beyond $T_{\text{stale}} = 30$ min superseded by the next writer—and within a cooldown $T_{\text{cd}} = 300$ s a same-direction rewrite is rejected while a reverse-direction correction stays allowed. Same-direction ratcheting is therefore prevented; alternating corrections remain allowed and are bounded only by the window, the step rule, and the backstop. Cross-agent in-window oscillation (two agents on one node) is out of the guardrails’ scope and listed as a limitation (Sec. VI-A). (3) *Soft interlock*. $v \in C(n, r)$ is checked before any driver interaction. (4) *Write with readback*. The value is encoded through t_n , sent, read back, decoded and compared under

$$\tau(n) = \max(0.5 \cdot 10^{-d(n)}, 0.005(n^{\max} - n^{\min})), \quad (2)$$

in engineering units—an engineering default, overridable per node in configuration: the larger of half a displayed least-count digit and 0.5 % of span ($d(n)$: display precision of n). The tolerance is expressed to the driver in raw units through t_n . A write is *ok* only if the decoded readback agrees within $\tau(n)$; a driver that cannot read back yields an explicit *degraded* flag, never a silent confirmation. The degraded write still executes and is journaled as unverified; whether no-readback actuators may take *auto* bindings at all is deployment policy—we recommend manual-only binding for blind actuators—and we return to this as a limitation (Sec. VI-A). (5)–(6) *Journal and audit*. Every executed write appends a signed anchor ($id, line, n, prev, new, src, a, r, approval, record, t$), and the audit plane receives a typed event attributed to the actor’s kind, deduplicated over 10 s.

Two safety invariants live on this path; Sec. IV-E adds the third:

- **I1 (containment)**. Every executed agent write satisfies $v \in C(n, r)$ (both branches). *Anchor*: single-entry write function. *Evidence*: rejection-path acceptance tests; live-instance verification (Sec. V-C).
- **I2 (readback closure)**. A write is reported *ok* only if the decoded readback lies within $\tau(n)$ or carries an explicit degraded flag. *Anchor*: Eq. (2) in the write executor. *Evidence*: deadband and calibration tests; the no-readback ablation.

D. Production-Line Operations Management

The framework models a production line as a four-level object tree—line, product, recipe, *run*—mapping onto the ISA-95 equipment hierarchy and an ISA-88-style *formula* binding (parameter limits bound to a run identity; procedural/phase control out of scope) [29], [30]. Every governance object hangs off this tree: the recipe window constrains writes jointly with the hard range while a run is active (Eq. (1)); acquisition

Algorithm 1 Governed write path $\text{WRITE}(n, v, src, a)$

Require: node n , value v , source src , actor a ; active run r if any

```

1: if  $src = \text{agent} \wedge \text{binding}(n) = \text{manual}$  then
2:   route to approval; return pending  $\triangleright$  Stage 0; lapse  $\Rightarrow$  never
   executed
3: end if
4: if  $\text{binding}(n) = \text{unbound}$  then
5:   return reject(FORBIDDEN)  $\triangleright$  tool surface never dispatches
   it
6: end if
7: if gateway paused  $\vee \neg n.\text{enabled}$  then
8:   return reject(CONFLICT)
9: end if
10: if  $src = \text{agent}$  then
11:   if  $\exists$  open record  $r'$  of  $a \wedge \text{age}(r') < T_{\text{stale}}$  then
12:     return reject(CONFLICT)
13:   else if  $\exists$  open record  $r'$  then
14:     SUPERSEDE( $r'$ )
15:   end if
16:   if same-direction write within  $T_{\text{cd}}$  then
17:     return reject(CONFLICT)
18:   end if
19: end if
20:  $W \leftarrow C(n, r)$   $\triangleright$  Eq. (1)
21: if  $v \notin W$  then
22:   return reject(soft interlock)
23: end if
24:  $v^{\text{prev}} \leftarrow$  current engineering value of  $n$ 
25:  $(raw, rb) \leftarrow$  driver.write( $n, t_n^{-1}(v)$ )
26:  $v_{\text{rb}} \leftarrow t_n(rb)$ ; if readback unavailable then  $degraded \leftarrow \text{true}$ 
27: if  $degraded$  then  $ok \leftarrow \text{false}$  else  $ok \leftarrow |v_{\text{rb}} - v| \leq \tau(n)$   $\triangleright$ 
   Eq. (2)
28: JOURNALAPPEND( $n, v^{\text{prev}}, v, src, a, r, \dots$ )  $\triangleright$  executed writes
   only; approval and record fields elided
29: AUDIT(write,  $n, v, a$ )  $\triangleright$  10 s dedup
30: return ( $ok, v_{\text{rb}}, degraded$ )
```

samples only nodes on lines with an active batch, while PLC-side protection persists between batches per P2; the backstop (Sec. IV-E) scores open records against the active window. An active run binds exactly one recipe version to one batch, giving every write a unique governance context: the window $W_r(p)$ of Eq. (1).

Operations monitoring rides the alarm chain. Each node’s state machine spans *ok/warn/alarm/offline*: entering *warn* outside bands placed 8 % of span inside each range limit (two-sided), with 2 % hysteresis, three-frame *ok* $\leftrightarrow$ *warn* debounce, watermark-based out-of-order defense, and recipe-window violations as edge-triggered *alarm*; acknowledgment is a human step with escalation (default 15 min). This implements the detection mechanics of the ISA-18.2/IEC 62682 lifecycle (rationalization and flood management out of scope) [23], [46]. Time series persist to TimescaleDB (batched 500 ms flushes, tiered retention), frames to MinIO with fallback to local disk, and platform state to SQLite (WAL, FTS5); every alarm, write, recipe change, and rollback row carries line/product/recipe dimensions for the metric slices of Sec. V, and both log surfaces—the central audit log with typed actor attribution (user, agent, system) and the operations log—are queryable views over the journal substrate of Sec. IV-C. Supervisory acquisition is gated on an active run: between batches the platform’s historian and alarm coverage pause while PLC-

side protection persists (P2)—a benchmark resource decision whose reduced-cadence monitoring variant is future work.

Rendering and commissioning share one surface: a *device-twin registry* persists, per scene entity, the bound 3D asset, the agent in charge (a virtual operator avatar), and the command set, alongside the three device-shadow state planes [47], [48] (*telemetry, desired, state*). A WebSocket bus broadcasts every reading, so the scene renders live values without polling; scene edits write back to the node registry, so the scene doubles as a commissioning editor that never actuates—*desired-to-state* reconciliation traverses the governed write path when it does.

E. Agent Closed-Loop Control

Fig. 2 shows the loop from the team’s point of view: the AgentTeam at the center, the four object families it operates on around it, and the numbered AgentLoop that turns plant data into governed setpoint changes.

One loop iteration, station by station. *Observe*: the bound agent queries fresh process values through `daq_query`, grounded by the semantic card’s unit, range, and freshness (Sec. IV-A). *Propose*: it issues `dcw_control` with a setpoint argument; the governed write opens an *optimization record*—the loop’s bookkeeping object, holding the record identity, node, binding mode, the frozen pre-write baseline (the parameter value preceding the record’s first governed write, with the statistics of the preceding $T_{bl} = 600$ s window), its open/closed state, and the verdict; one record per agent scope—and the proposal obeys the step rule (any single step $\leq 2\%$ of span). *Govern*: the stage checks and, on manual bindings, the approval gate apply exactly as on any write (Secs. IV-C and IV-G). *Actuate*: the driver writes and reads back. *Verify-judge*: a separate tool, `dcw_judge`, inspects the record against fresh acquisition data and returns *keep*, *rollback*, or *uncertain*—judgment is not execution: the verdict changes no PLC value, and only a separately invoked rollback actuates, itself traversing the full pipeline; an *uncertain* verdict leaves the record open for the operator. *Learn*: outcomes land in the team’s memory (Sec. IV-B), closing the informational loop.

Two bounds make the loop safe to leave running. First, the guardrails of Sec. IV-C: one open record per agent, stale records superseded at $T_{stale} = 30$ min, and the same-direction cooldown $T_{cd} = 300$ s. Second, the *backstop*: an open record under an *auto* binding is re-evaluated every $T_{re} = 30$ s once older than $T_{win} = 120$ s, judged against the active window $W_r(p)$ (falling back to $S(n)$) over every sample acquired since it opened; worst-case detection-to-rollback is bounded by $T_{win} + 2T_{re}$ plus the rollback write latency. If the out-of-window count reaches $B = 3$, the system issues a rollback verdict and executes it only while the node’s executed-rollback count is below $K = 2$; otherwise the verdict is recorded and escalated to a human. The rollback target defaults to the recorded baseline (finer granularities are operator-invoked, Sec. IV-F); every rollback write traverses this same pipeline, and a paused gateway defers the actuation—the fail-safe direction. Because the counter is per node and monotone, K is a node-lifetime cap. The third invariant:

- **I3 (bounded autonomy)**. Automatic rollbacks per node are at most $K = 2$ over the node’s life; the third

rollback attempt and all manual-policy rollbacks require human confirmation, and stale records are superseded, never accumulated. *Anchor*: the per-node rollback counter guarding the backstop. *Evidence*: rollback-manager acceptance tests; the backstop drill executes one system-initiated rollback end-to-end, and the escalation-at- K branch is exercised at acceptance-test level (Sec. V-E).

These bounds are stated over the pipeline, not over the LLM: whatever the agent proposes, the accepted-action language is constrained. This is the precise sense in which the system’s autonomy is *governed*.

F. Recipe Management and Rollback

The recipe lifecycle is versioned and non-destructive: editing a recipe *creates* version $n+1$ rather than overwriting; an operator can freeze a *known-good* baseline; *known-good rollback* re-dispatches the frozen parameter set to the PLC; and one-click *apply* starts a fresh batch under the chosen version. While a run is active, the bound version supplies the window $W_r(p)$ of Eq. (1), so recipe management and write governance cannot drift apart. Rollback has four granularities—the record baseline (the backstop’s default), a stable anchor, the batch start, and the known-good snapshot—all reached through the same governed write path and all journaled with full attribution. Off-window excursions therefore have exactly two bounded exits: automatic rollback while the executed-rollback count is below $K=2$ (Sec. IV-E), then mandatory human escalation.

G. HITL: The Human Control Boundary

The human boundary is a property of the *binding model*, not of a prompt. Binding a node to an agent selects one of three postures: *manual*—every agent write is routed to a human approval request and is not submitted further until a human decides; *auto*—writes proceed under interlock and backstop without the gate; *unbound*—no write path exists (the tool surface never dispatches one). An engineer binds nodes to a single agent or a whole team; the binding injects the semantic card into the agents’ context and derives their tool surface (Sec. IV-A).

The approval gate is engineered against the known failure modes of human-in-the-loop supervision [28], [49], [50], [51]: automation bias and complacency are countered by deduplication per node and record and by lapse (an unapproved request lapses with its record rather than queueing indefinitely—lapse is record-bound; no independent timer runs); out-of-the-loop degradation is countered by keeping acknowledgment and escalation human. Approval is *decoupled from execution*—every pipeline stage re-runs after the decision, so a recipe change during deliberation cannot be exploited through a stale approval. Operators approve from the web console or the TUI workbench, which share the same approval queue, and each decision lands in the journal anchor of the resulting write (Sec. IV-C), so human decisions are attributable exactly like machine ones. The human-facing approval card itself (proposal against live value, window position, step-rule and baseline context, rationale) is productization surface: the journal and

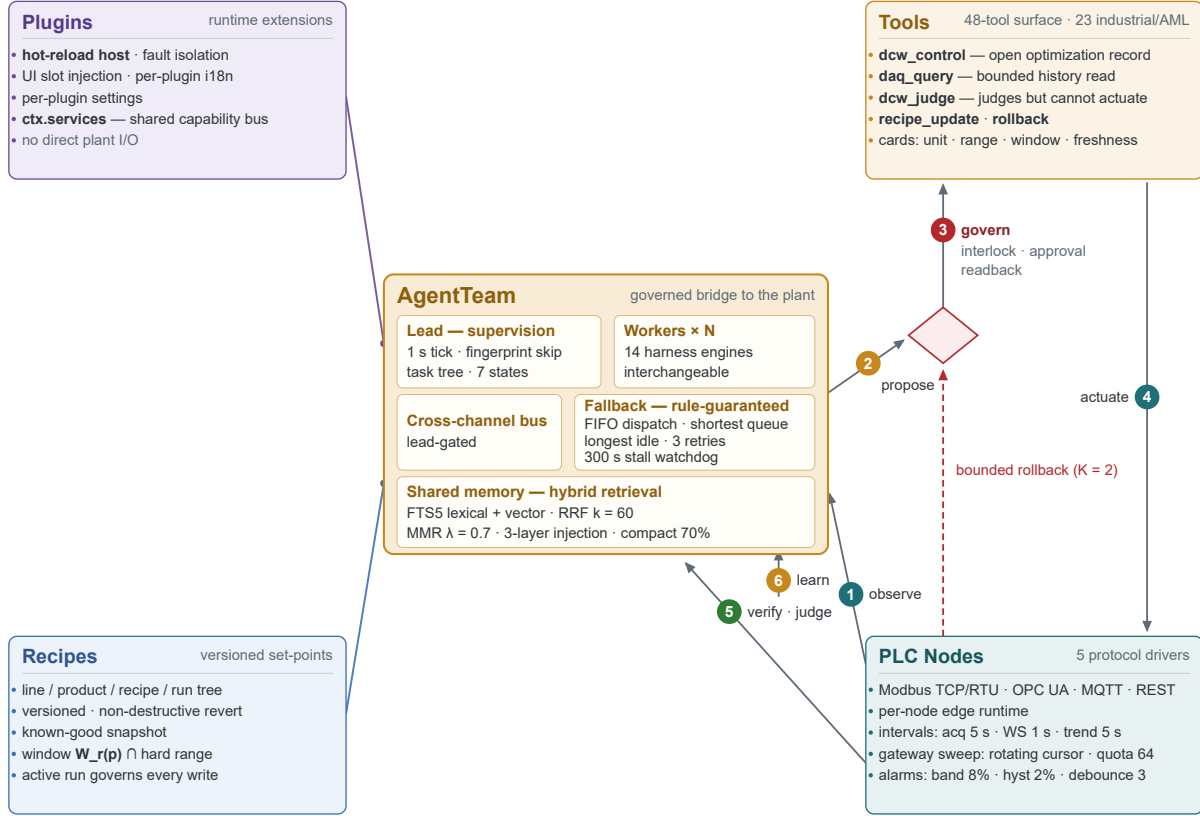


Fig. 2. AgentTeam-centric view of the closed loop. **Center:** the team’s internal design—lead supervision with a rule-guaranteed fallback, N harness-backed workers (14 interchangeable engines), a lead-gated cross-channel bus, and hybrid episodic–semantic memory. **Spokes:** the industrial tool surface (48 host tools, 23 industrial/automated-ML), the plugin host (runtime extensions without direct plant I/O), PLC nodes (five protocol drivers with per-node edge runtimes), and versioned recipes. **Numbered edges:** the AgentLoop—observe (1, daq_query) → propose (2, dcw_control opens an optimization record) → govern (3, interlock–approval–readback, Sec. IV-C) → actuate (4) → verify–judge (5, dcw_judge keeps or rolls back) → learn (6, memory harvest); the dashed return edge is the bounded rollback ($K=2$).

semantic card define its data, and its ergonomics we scope as future work (Sec. VI-A).

Three further separations bound what agents can do: *judgment is not execution* (Sec. IV-E); *lead-only tools* (team-level operations, cross-channel messaging) are restricted to lead agents with approval; and alarm *acknowledgment* remains a human step with escalation (Sec. IV-D), as does the escalation-at- K branch of the backstop. The gate governs the sanctioned path only. Harness engines are privileged local processes—a compromised engine could write to the protocol stacks directly, the failure mode *that* indirect prompt injection targets [52]—so engine containment is a *deployment* property, not a measured guarantee: the deployment model combines an availability allow-list probe (Sec. IV-B), a least-privilege service account, and IEC 62443 zoning with OT-security guidance [40], [53], [54] to keep the governed surface the only *reachable* write path, and no injection drill or privilege audit appears in Sec. V. Inside the sanctioned path a prompt cannot *enlarge* an agent’s authority; it can only abuse authority the operator already granted, which the approval gate and the chain limit bound. Governance is fail-safe under storage degradation (F6).

V. BENCHMARK EVALUATION

A. Benchmark Protocol and Data Layers

AW-INDUSTRIALBENCH is governed by three principles. *Determinism:* a seeded PRNG (mulberry32) and a scenario-snapshot API make structural metrics (interception verdicts, judge-class checks) point-wise reproducible; objective values over *sampled* process variables remain timing-sensitive (Sec. V-I); the offline grid-search optimum W^* is the reference bound. *Layered realism:* results are reported per data layer (Table I), never pooled. *Executable protocol:* one released audit harness (bench/run.mjs, bench/pipeline.mjs) executes every check of this section—nothing is asserted by hand.

The workload is phrased as five task families— $T1$ closed-loop setpoint correction in-window, $T2$ out-of-window excursion, $T3$ multi-node joint adjustment, $T4$ batch parameter recovery, $T5$ mid-run objective change—and six fault families injected at the simulator control API: F1 step drift, F2 sensor freeze, F3 communication interruption, F4 out-of-order injection, F5 out-of-window writes, F6 storage unavailability. F5 stresses containment (I1, Sec. IV-C); F2/F3 are exercised below. The measured baseline is $B1$, the pipeline’s own ablation via env-gated bench-mode-only bypass switches (Sec. V-G);

TABLE I
DATA-CREDIBILITY LAYERS; CLAIMS ARE LABELED BY THE HIGHEST
LAYER MEASURED.

Layer	Description	Status
L1	Seeded simulation: two independently defined plant scenarios (strategy-signal film line; ODE cast-film twin), seeded noise, offline optimum W^*	measured
L2	TEP [33] / SWaT [34] replay through real drivers	designed
L3	One physical OPC UA/Modbus device on the real stack; write-readback latency	designed
L4	Digital-twin case study: seeded closed-loop campaign on the cast-film line—deterministic controller over three seeds and an agent team over four runs (Secs. V-F, V-H)	measured

further baseline groups—general agent frameworks, human operators, classical control, cross-engine replication—are defined in AW-INDUSTRIALBENCH’s released protocol and are *not* measured here.

Metrics form three families. *Safety*: out-of-window write interception rate (denominator = seeded attacks), false-block rate (denominator = legal in-window writes attempted), and per-write attribution rate (target 100%). *Performance*: the *automatic segment* (proposal acceptance to verified write), kept separate from human-approval waiting time so human response is never conflated with system latency. *Cost/quality*: task goal rate, tokens and LLM calls per task, approval wait time, human interventions per task—declared here; measured cost reporting is deferred to the released campaign archive. Coverage as measured below: F5 and the I1/I2 probes are fully measured; F3 (link interruption) appears as a drill and an F2-class freeze is drilled as a *process* freeze (F2 is defined as sensor freeze—the semantic card’s freshness path remains untested); F1, F4, F6 and the escalation-at- K branch are exercised only at acceptance-test level or unmeasured (Sec. VI-A); T1/T2 are measured directly, T3–T5 remain protocol vocabulary. *Reproducibility protocol*: every run records its seed, harness source hash, and commit, appends to never-overwritten timestamped directories, and is compared against frozen baselines by a gate that checks judge-class equality (Sec. V-I); released LLM arms pin model versions, and temperature-0 applies to campaign configurations. The plant simulator and the platform are booted by the pipeline itself; all measurements ran on one host (Ryzen 7 9700X, 31 GB RAM, Windows), and one command reproduces the section: `node bench/pipeline.mjs --profile integrated --seed 42 --cl-seeds 3`.

B. Cross-Scenario Portability (Framework Claim)

The central framework claim—adapting the deployment to a new line is a configuration task, not an integration project—is measured directly by the pipeline’s final stage. Everything is first provisioned on the *cast-film physics* scenario (ODE plant model, six actuators, seven sensors, five protocol drivers, vector and image frames); the plant then switches to a second, independently defined scenario, *film-line*: a coating-oven line with a disjoint signal domain and per-signal strategies rather than a physics engine. The identical five-step flow (export,

connection test, node creation, line/product/recipe provisioning, agent binding) re-instantiates the deployment on both, over the real five-protocol transports of Sec. V-D’s simulator stack, because driver configurations derive from each device’s export (P3; Sec. IV-A).

Measured on the second scenario: five devices across all five protocol drivers are commissioned in one pass (the artifact is five derived driver configurations plus one scenario identifier); three writable-setpoint devices become governed lines (Modbus TCP, OPC UA, MQTT) and the two acquisition-only devices attach as satellites; all five nodes stream live data (8–12 one-second buckets in the quoted pair); governed writes are accepted with readback deviation $\leq 0.04^\circ\text{C}$ on Modbus TCP and OPC UA (exactly zero on OPC UA in the quoted run); and all nine seeded out-of-window attacks are rejected with zero false blocks (none of the 3 legal in-window writes blocked). The structural verdicts—100 % interception, 0 % false blocks—are identical across scenarios: per-line engineering moved into authoring the scenario definition, and the platform absorbed the difference unchanged. These nine attacks, pooled with the twenty five-protocol attacks of Sec. V-D, compose the abstract’s 29/29; the six mock-tier attacks of Sec. V-C are a separate in-process pool. The same four commissioning surfaces are shown as an operator drives them in the released walkthrough artifact: node creation from the device export, twin and recipe-window binding, channel deployment, and the optimization record beside the setpoint ledger—no plant-specific code on any surface.

C. Static Audit and Dynamic Invariant Verification on a Live Instance (L1 Tier)

Before any dynamic campaign, the audit harness verifies the artifact at source level. Three checks run offline: a *capability inventory* against the claims of Sec. III; a *paper-code consistency* check that locates 20 quantitative constants cited in this paper (sweep quota 64, breach threshold 3, rollback-chain limit 2, RRF $k=60$, MMR $\lambda=0.7$, compaction 0.7, watchdog 300 s, hysteresis 2 %, among others) at their implementation anchors; and a *stage-ordering* check asserting that Algorithm 1’s governed-write stages appear in the declared order at their source anchors. All pass: 20/20 anchors, 2/2 ordering chains—engineering discipline rather than validation evidence, since the harness audits the artifact of which it is part.

The rest of this subsection verifies behavior: L1-tier evidence (Table I), driving a pinned platform instance (v0.7.39, mock drivers) exclusively through its REST API with seeded randomness (mulberry32, seed 42). A benchmark line is created with global safety range $[120, 260]^\circ\text{C}$ and a batch started with active recipe window $[175, 205]^\circ\text{C}$; the invariants of Sec. IV-C are then attacked and probed, including the decision boundaries and both branches of Eq. (1).

I1 (containment). Six seeded gross out-of-constraint writes—global-range violations (107, 269), window violations while the batch is active (219, 163), and extreme/negative values (10^6 , -25)—are all rejected with typed validation errors, while all three in-window control writes are accepted:

100 % interception at 0 % false blocks. Boundary probes at $\pm$ one least-count behave as specified (205.0 accepted; 205.1, 174.9 rejected), and the rejection message cites the window as the binding constraint. With the batch stopped the window branch deactivates and the hard range alone governs: an out-of-range write (300) remains rejected while a within-range, out-of-window write (130) is accepted—the soft interlock is a batch-scoped constraint, not a constant.

I2 (readback closure) and attribution. All accepted writes return a driver readback within the tolerance of Eq. (2) ($\tau = 0.7^\circ\text{C}$ at one decimal digit and a 140°C span) and produce journal anchors with source, actor, timestamp, and previous/new values—100 % attribution. Acquisition held the 1 s cadence, and the samples API answered 20 probes at p50 6–22 ms / p95 8–63 ms.

Reproducibility. Every execution on a pinned harness (Secs. V-I and V-G) produced identical judge-class metrics—the same seeded attacks, rejections, boundary verdicts and ledger outcomes; only fixture identifiers and latency varied. `bench/compare.mjs` asserts judge-class equality between a fresh run’s record and the archived baseline (REPRODUCIBLE; the gate rejects a tampered metric, validated by a self-test). The Wilson 95 % interval for 6/6 trials ([61 %, 100 %]) is the appropriate reading of any small- n rate.

Scope. These checks exercise the accepted-action language at the API boundary with the LLM out of the loop—the invariants are defined over the pipeline, not over any engine (P4, Sec. IV-C).

D. Real-Protocol Plant Scenarios (Five-Protocol PLC Node Simulator)

The API-tier checks of Secs. V-C and V-G use in-process mock drivers; this subsection re-runs the working-condition scenarios over *real protocol stacks* against an independent five-protocol PLC node simulator (Modbus TCP/RTU slaves, OPC UA server, MQTT broker/publisher, HTTP endpoints, ODE plant model). Driver configurations derive at runtime from the simulator’s integration-export API—no hand-written connection settings—and every candidate driver passes the connection test before a node is created.

The pipeline provisions one production line per device that exposes a writable setpoint (4 lines: Modbus TCP, OPC UA, MQTT, HTTP) and attaches the acquisition-only device (a Modbus RTU gel counter—a defect-counting photoelectric sensor) as a *satellite* node, because acquisition samples only nodes whose line has an active batch. All five nodes stream live data through their real transports (2–14 one-second buckets in the settle window, Table II). A governed setpoint write costs 19–112 ms (p50) by protocol, the Modbus TCP transaction dominating (its p95 reflects the first write’s connection setup); readback closes within 0.04°C on Modbus TCP and exactly on OPC UA, while the MQTT and HTTP control channels report no readback by design (publish-only / echo-only actuators)—a driver property the write path records rather than conceals. Five seeded attacks are rejected on *every* writable protocol (5/5 each; 20/20 pooled, a set-coverage statement over the seeded suite rather than

TABLE II
REAL-PROTOCOL SCENARIOS OVER THE FIVE-PROTOCOL PLC NODE SIMULATOR (L1, SEED 42). FOUR LINES PLUS ONE ACQUISITION-ONLY SATELLITE. ACQ. PTS: SAMPLES IN THE SETTLE WINDOW. WRITE LATENCY: P50/P95 OVER SIX IN-WINDOW WRITES PER LINE. “N/A”: PUBLISH-ONLY (MQTT) / ECHO-ONLY (HTTP) ACTUATORS REPORT NO READBACK. F5: SEEDED OUT-OF-WINDOW WRITES REJECTED, OF 5 PER PROTOCOL.

Protocol	Acq. pts	Write p50/p95 (ms)	Readback	F5
Modbus TCP	2	111.5 / 141.6	0.04°C	5/5
OPC UA	11	19.0 / 22.1	exact	5/5
MQTT	12	19.4 / 19.5	n/a	5/5
HTTP	12	32.8 / 33.2	n/a	5/5
Modbus RTU [†]	14	—	—	—

[†]acquisition-only satellite node (gel counter).

an independence-based rate) with zero false blocks on the four legal in-window writes, and the deterministic tool-level control loop (`daq_query`→`dcw_control`→`dcw_judge`) converges on all four lines. A link-interruption drill (simulator service stopped and restarted) is detected and recovered, and a process-freeze injection shows defense in depth: the governed write still passes (readback verifies the register, not the process) while the independent recipe-window monitor catches the failure within 2 s—the layer separation of Secs. V-C and IV-C.

E. Governance Surface: Recipe Lifecycle, Rollback, and Approval

With the LLM out of the loop, each probe below ran end-to-end in one pipeline run against the governance stack’s *lifecycle objects*. *Non-destructive history*: a revert creates version 3 rather than overwriting. *Judge-execute separation*: judging `rollback` changed no PLC value (196.60°C throughout)—only the separately invoked rollback restored the origin value (203.40°C); an agent’s verdict is an intent record, not an actuation. *Bounded autonomy in action (backstop drill)*: freezing the process variable out of window under an open auto-binding record triggers the backstop—the record is judged `rollback by the system` and the PLC restored to the baseline ($186.2 \pm 0.75^\circ\text{C}$), cadence-bound at 120–130 s across releases; chain limit $K=2$ remains an engineering cap (Sec. VI-A). *Default non-execution*: in manual mode an agent write blocks until the approval releases it (148.2 rpm on OPC UA in the quoted run; poll-inclusive decision latency ranged 6–468 ms across released runs); the journal holds ten anchors over four sources and both log surfaces return full 200-entry windows.

Platform subsystems: vector and image frames from the profile and CCD channels are stored as first-class acquisition objects; a mock lead dispatches a child task that a worker completes (parent COMPLETED); duplicate team-memory writes under one deduplication key yield one entry; the registry enumerates 14 engines, with environment-available counts of 12–14 across releases (one run’s probe glitched to zero and is excluded).

F. Closed-Loop Optimization Benchmark

The closed-loop benchmark quantifies how much of the offline optimum a *governed* loop recovers on the cast-film

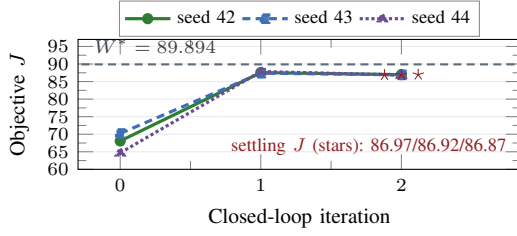


Fig. 3. Governed closed-loop optimization on the cast-film twin (three seeds, real transports, deterministic controller). J is the objective of Eq. (3); all seeds converge in two iterations to 97.0–97.2 % of the offline grid optimum W^* (dashed). Red stars mark the iteration-2 settling measurements (offset horizontally for visibility); the quoted ratios are the J_{end} means of both evaluations over W^* .

twin. The pipeline instantiates all six actuators (zones on Modbus TCP, screw on OPC UA, line speed on MQTT, die gap on HTTP), starts a batch off-optimum ($N=150$ rpm, $v=95$ m/min, zone 200°C), and runs a controller that reads through the acquisition path and writes through the governed path. The objective

$$J = 55 J_{\text{thick}} + 25 J_{\text{quality}} + 8 J_{\text{energy}} + 7 J_{\text{throughput}} \quad (3)$$

constrains melt temperature to $[195, 225]^\circ\text{C}$ and pressure to ≤ 22 MPa. The controller is model-light and fixed in advance: it inverts $h \propto N/v$ at constant line speed ($N \leftarrow N \cdot (50/h)$, ± 22 rpm per step) and corrects the heater setpoint only when melt temperature leaves $[196, 224]^\circ\text{C}$; it contains no LLM, so the benchmark isolates the *governed loop* rather than a model. Convergence requires two consecutive evaluations with $h \in [48, 52] \mu\text{m}$, defect $< 2\%$, pressure ≤ 22 MPa.

Fig. 3 reports the trajectories; J_{end} is the mean of the two consecutive in-band evaluations. From $J_0 = 64.7\text{--}70.2$ the loop reaches $J_{\text{end}} = 87.2\text{--}87.4$ against $W^* = 89.894$ (weights unnormalized, 0–95 scale; real-time physics; offline grid resolution $2.5^\circ\text{C} \times 5$ rpm $\times 2.5$ m/min, die gap fixed at 1.0 mm), i.e. 97.0–97.2 % of the optimum in the quoted run, converging in two iterations with at most two governed writes per seed, no rejected write and no constraint violation; mean wall time is 36 s per seed. Thickness moves from 55.2–56.1 μm (target $50 \pm 2 \mu\text{m}$), defect settles at or below 0.64 %, pressure at 16.6–16.9 MPa, melt temperature stays inside $[195, 225]^\circ\text{C}$. Per-seed values in the quoted run ($J_0 \rightarrow J_{\text{end}}$, ratio): seed 42: 68.07→87.26 (0.971); seed 43: 70.21→87.16 (0.970); seed 44: 64.69→87.37 (0.972). Across all completed releases of the same command and seed (three early-development and two interrupted runs excluded), per-seed J/J^* spans 0.962–0.973 and run-level means 0.965–0.971; the residual spread reflects the sampled process variables from which J is computed. The 3–4 % shortfall is a throughput/energy trade, not a control failure: the grid optimum sits at a low-throughput corner of the objective, while the loop holds the higher line speed it started from.

G. E1a: Pipeline-Tier Governance Ablation

Using the env-gated bench-mode bypass (production builds refuse the flag), we compare the full pipeline

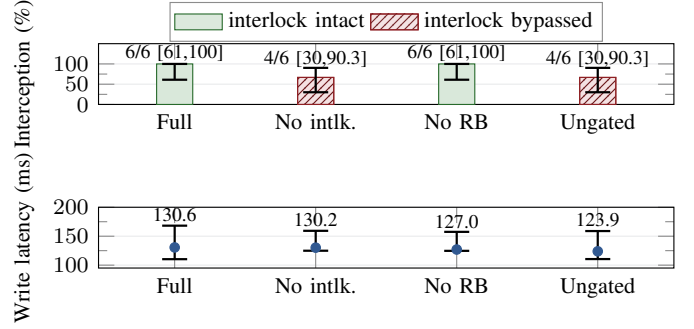


Fig. 4. E1a four-arm governance ablation (mock drivers, seed 42; nine repetitions per arm across three released runs, judge-class values identical). Top: interception per arm; error bars are Wilson 95 % intervals over the six seeded attacks, with endpoints printed—structural rather than statistical at $n=6$. Bottom: mean p50 write latency per arm as point ranges, not bars, so a truncated axis cannot exaggerate differences; whiskers span the minimum observed p50 to the maximum observed p95, and arm means are printed. Aggregates recomputed from the released per-repetition CSVs by `bench/tools/fig4_analyze.py`.

against the three ablation arms of B1—soft interlock bypassed (no-interlock), readback verification bypassed (no-readback), and fully ungated—on the live instance (mock drivers, seed 42, three repetitions per arm per released run, fresh fixture per repetition). The hard range is structurally enforced at the driver layer, so the ablation isolates exactly the two bypassable mechanisms. Fig. 4 reports the result; values were identical across repetitions. Three findings: (i) bypassing the interlock collapses interception from 6/6 to 4/6 (identically in the no-interlock and ungated arms)—the window-class attacks execute while global-range attacks remain structurally rejected; (ii) the executed breaches are fully journaled (twelve out-of-window anchors per bypass arm per run: six breach writes plus six accepted edge probes): *attribution is not prevention*; (iii) no arm adds latency resolvable at this tier (p50 110–141 ms, p95 150–169 ms across all arms; an equivalence claim from overlapping ranges, no test applied). Boundary behavior follows each arm’s specification (9/9 probes; Wilson 95 % $[70, 100]\%$), false blocks are 0/3 in every arm, and the no-readback arm matches Full on in-memory drivers (the mock readback echoes the command), which is why its verification value is exercised by the frozen-process injection above.

H. Case Study: LLM-Agent Closed-Loop Optimization (L4)

The benchmark of Sec. V-F isolates the governed loop under a fixed controller; this subsection reports the complementary result—the same plant and objective closed by an LLM agent team. Four campaigns are released (first-party omp engine driving glm-5.3-flash in a lead-supervised team; manual-mode bindings; physics accelerated $6\times$; seed 42; plant and objective as in Sec. V-F). Their sampling settings predate the temperature-0/model-pinning discipline of AW-INDUSTRIALBENCH’s released protocol (harness defaults), one reason we report them as an existence proof rather than a controlled measurement. In each campaign the team opened an optimization record, probed the plant with `daq_query`,

and actuated through `dew_control` over real protocol transports; the write ledger records 7, 7, 15 and 10 governed writes respectively with per-write journal anchors, and approval decisions recorded by a scripted operator emulation—an autonomous-campaign configuration, not a production posture.

The outcome is unfavorable to the agent. The agent objective reached $J_{\text{agent}} = 67.7, 31.2, 68.3, 68.5$ against $W^* = 89.894 - 0.75, 0.35, 0.76, 0.76$ of the offline optimum, median 0.76—and the thickness window $50 \pm 2 \mu\text{m}$ was *not* reached: three campaigns settled at a $55.33 \mu\text{m}$ window mean and the fourth produced no in-window samples (its MQTT thickness channel also refused connection), while the melt-pressure node failed its connection test on OPC UA in all four. Per-campaign initial objective values and itemized failed checks are preserved in the released campaign archive; the 0.35 tail is partly infrastructure (the refused channel), not purely decision quality.

This is a measured negative with a design implication: an LLM team drove real protocol transports through the governed path with every write reconstructable, yet on this objective it does not match a two-step fixed controller (0.76 vs. 0.97 of W^*)—the propose/dispose division of roles working as intended: the agent explores and proposes; the deterministic path converts proposals into converged actuation.

I. Reproducibility and Threats to Validity

Re-running the whole pipeline twice with an identical harness (source hash 204fc9ee; runs 20260914152838-fwg and 20260914153400-ce0), seed, and commit yields *bit-identical* structural results: 55/55 pipeline checks pass in both runs; every seeded attack is rejected with zero false blocks on *both* plant scenarios (9/9 on the second); all four per-line loops converge; the portability and backstop-drill outcomes are identical; and $W^* = 89.894$. The one structural difference is benign: governed-write totals are 6 vs. 5 (seed 44 converged on its first write)—“at most two governed writes per seed” still holds. What varies is what the abstraction predicts: governed-write latency is environment-dependent, and the closed-loop objective—computed from *sampled* process variables—lands in $J/J^* \in [0.962, 0.973]$ across the same campaign repetitions as Sec. V-F.

Threats to validity. *Construct:* success oracles are database- and readback-defined, mitigating self-reported success bias. *Internal:* the released protocol requires temperature-0 and pinned model versions, but the four reported campaigns predate that discipline (Sec. V-H)—one of their confounds; no nonparametric comparisons are reported, and the latency-equivalence claim in Sec. V-G rests on overlapping ranges. *External:* all measured layers are simulation-tier (two seeded plant scenarios, bounded by the ODE twin’s fidelity per scenario); public-dataset replay (TEP, SWaT) is designed but unmeasured; the four LLM campaigns share one seed and carry no cross-seed variance estimate. Results are layer-labeled (Table I) and we claim no transfer beyond the measured layers (L1, L4) without per-layer evidence. *Conclusion:* the closed-loop result uses a fixed, pre-declared controller and three

plant seeds; the LLM campaign is a measured negative, not a headline.

VI. DISCUSSION AND LIMITATIONS

A. Scope and Limits

The pipeline constrains the *action language* of agents, not the quality of their choices: proposals still derive from LLM competence, context quality, and tool ergonomics, and the unbounded statistical components (LLM decisions, with deterministic rule fallbacks) bottom out at the rules’ behavior. Bounded mechanisms—interlock, readback, chain limit—absorb bad proposals. The backstop’s $K=2$ caps a node’s lifetime automatic rollbacks, an engineering limit exercised only up to one in the drill. The invariants borrow IEC 61511’s separation of basic control from safety functions [41] but remain engineering properties, not certified safety functions: the by-construction guarantee holds within the sanctioned path of Sec. IV-G; AGENTWORKSHOP does not replace certified instrumentation. The completed evidence covers the artifact audit, live-instance invariant verification, a dynamically exercised rollback, real-protocol scenarios on both scenarios, the pipeline-tier ablation, and one plant domain at L1/L4; the remaining agent-in-the-loop baselines (general frameworks, human operators, cross-engine replication) are defined in AW-INDUSTRIALBENCH’s released protocol rather than reported here—the measured evidence covers *governed supervisory control*, not open-ended plant engineering. Three guardrail scopes are likewise declared rather than closed: cross-agent in-window oscillation is out of the per-scope guardrails’ reach; fault families F1/F4/F6 and the escalation-at- K branch are exercised only at acceptance-test level or unmeasured; and binding policy for no-readback actuators is deployment advice, not enforced code. Deployment/administration layers that real plants add are likewise future work: approval-card ergonomics, lapse timers and agent re-proposal backoff, binding-change (manual→auto) governance, K -counter reset policy, multi-operator queue roles and shift handover, and approval-load budgeting against alarm-management practice [23], [46]. The integration-cost claim is correspondingly scoped: the 16 s result measures recommissioning once a scenario definition exists; authoring one—including the expert-validated recipe windows—is engineer effort we do not price, and a mis-authored window is itself a configuration hazard. Workforce and regulatory placement (the operator drifting into an exception-handler role; who signs off an LLM-proposed setpoint under management-of-change) deserve treatment beyond this paper’s scope. Three lessons may transfer: single-entry writes are the keystone; judgment/execution separation pays twice; deterministic rehearsal against an offline-optimum oracle surfaces defects before hardware.

VII. CONCLUSION

We presented AGENTWORKSHOP, an integration framework embedding LLM multi-agent teams natively into the supervisory fabric of protocol-heterogeneous production lines—one process, one data domain, physical semantics instead

of register addresses, team-level node binding, configuration-only commissioning—with AW-INDUSTRIALBENCH, a reproducible evaluation protocol where every agent-issued write passes through a single governed pipeline defining three safety invariants over the accepted-action language, independent of engine. A governed closed-loop benchmark on a seeded cast-film line recovers 97.0–97.2 % (quoted run) of the offline grid optimum in two iterations and at most two governed writes per seed (per-seed $J/J^* \in [0.962, 0.973]$ across released runs); the backstop’s rollback is exercised end-to-end on a live breach; and four uncontrolled same-seed LLM-agent campaigns actuate the same loop over real transports with every write attributed, reaching 0.35–0.76 (median 0.76) of the optimum (propose/dispose in action). The remaining baseline campaigns are defined in AW-INDUSTRIALBENCH’s released protocol; the platform source (PolyForm Noncommercial), the benchmark harness with frozen baselines and run archives, and the PLC node simulator (git submodule) are versioned in the project repository,¹

ACKNOWLEDGMENTS

The authors thank the open-source communities whose tooling supported this work. AI-based assistants were used for language editing, code-assisted analysis, and figure preparation under continuous human direction; the authors reviewed and edited all content and take full responsibility for the work.

REFERENCES

- [1] M. Raza *et al.*, “Industrial applications of large language models,” *Scientific Reports*, vol. 15, 2025, art. no. 13755.
- [2] S. Colabianchi, F. Costantino, and N. Sabetta, “Assessment of a large language model based digital intelligent assistant in assembly manufacturing,” *Computers in Industry*, vol. 162, 2024, art. no. 104129.
- [3] J. Zhang, H. Wang, Z. Jia, J. Dong, and L. Ren, “CoMA-IKG: LLM-driven multiagent framework for automated construction of industrial knowledge graph,” *IEEE Transactions on Industrial Informatics*, vol. 22, no. 6, pp. 5553–5564, 2026.
- [4] Z. Liu, R. Zeng, D. Wang, G. Peng, J. Wang, Q. Liu, P. Liu, and W. Wang, “Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents,” *IEEE Transactions on Software Engineering*, vol. 52, 2026, arXiv preprint arXiv:2410.14209.
- [5] M. Fakihi, R. Dharmaji, Y. Moghaddas, G. Quiros Araya, O. Ogundare, and M. A. Al Faruque, “LLM4PLC: Harnessing large language models for verifiable programming of PLCs in industrial control systems,” in *Proc. 46th Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2024, arXiv preprint arXiv:2401.05443.
- [6] F. Ren *et al.*, “MetaIndux-PLC: A control logic-guided LLM for PLC code generation in industrial control systems,” *Applied Soft Computing*, vol. 184, 2025, art. no. 113673.
- [7] Y. Xia, D. Dittler, N. Jazdi, H. Chen, and M. Weyrich, “LLM experiments with simulation: Large language model multi-agent system for simulation model parametrization in digital twins,” in *Proc. IEEE ETFA*, 2024, arXiv preprint arXiv:2405.18092.
- [8] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, “Leveraging LLM agents and digital twins for fault handling in process plants,” in *Proc. IEEE ETFA*, 2025, arXiv preprint arXiv:2505.02076.
- [9] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, “Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA,” *arXiv preprint arXiv:2510.11300*, 2025.
- [10] J. Vyas, M. S. Gill, A. Markaj, F. Gehlhoff, and M. Mercangöz, “From detection to action: Using LLM agents for fault-tolerant control,” *arXiv preprint arXiv:2606.28011*, 2026.
- [11] P. Liu, L. Qian, X. Zhao, and B. Tao, “Joint knowledge graph and large language model for fault diagnosis and its application in aviation assembly,” *IEEE Transactions on Industrial Informatics*, vol. 20, no. 6, pp. 8160–8169, 2024.
- [12] Y. Ren, Y. Liu, T. Ji, and X. Xu, “AI agents and agentic AI—navigating a plethora of concepts for future manufacturing,” *Journal of Manufacturing Systems*, vol. 83, pp. 126–133, 2025.
- [13] B. Chen, L. Dai, C. Wang, J. Wan, L. Wang, F. Xiang, Z. Wang, X. Xia, M. Guizani, and G. Fortino, “Large AI models empowering intelligent manufacturing: Architecture, evolution, and prospects,” *IEEE Access*, vol. 14, 2026.
- [14] T. Cacciapaglia, D. Ruffino, D. Naso, and P. R. Massenio, “Automated PLC programming using a multi-task LLM system,” in *Proc. Int. Conf. on Control, Decision and Information Technologies (CoDiT)*, 2026.
- [15] C. Liu *et al.*, “LLM-enhanced embodied multi-agent manufacturing system,” *Journal of Manufacturing Systems*, vol. 88, 2026.
- [16] Z. Zhao, D. Tang, C. Liu, L. Wang, Z. Zhang, H. Zhu, K. Chen, Q. Nie, and Y. Jia, “A large language model-based multi-agent manufacturing system for intelligent shopfloors,” *Advanced Engineering Informatics*, vol. 69, 2026, art. no. 103888.
- [17] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, “Agent-based supervision for service-oriented industrial cyber-physical systems,” *IEEE Transactions on Industrial Informatics*, vol. 21, no. 3, 2025.
- [18] L. Yang, S. Luo, X. Cheng, and L. Yu, “Leveraging large language models for enhanced digital twin modeling: Trends, methods, and challenges,” *arXiv preprint arXiv:2503.02167*, 2025.
- [19] R. Wu, J. Ai, and T. Li, “InstructMPC: A human-llm-in-the-loop framework for context-aware control,” *arXiv preprint arXiv:2504.05946*, 2025.
- [20] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, “Autonomous chemical research with large language models,” *Nature*, vol. 624, pp. 570–578, 2023.
- [21] S. H. Vemprala, R. Bonatti, A. Buckner, and A. Kapoor, “ChatGPT for robotics: Design principles and model abilities,” 2023.
- [22] G. Yoon, S. Lee, and J. Y. Sim, “ModuLoop: Low-level code generation using modular synthesizer and closed-loop debugger for robotic control,” *IEEE Robotics and Automation Letters*, 2025, early access, doi: 10.1109/LRA.2025.3623437.
- [23] International Society of Automation, “Ansi/isa-18.2: Management of alarm systems for the process industries (iec 62682),” Standard, 2016.
- [24] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, “ τ -bench: A benchmark for tool-agent-user interaction in real-world domains,” *arXiv preprint arXiv:2406.12045*, 2024.
- [25] Y. Xu *et al.*, “Theagentcompany: Benchmarking LLM agents on consequential real world tasks,” *arXiv preprint arXiv:2412.14161*, 2024.
- [26] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, “Control barrier function based quadratic programs for safety critical systems,” *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.
- [27] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, “Safe reinforcement learning via shielding,” in *Proc. AAAI Conference on Artificial Intelligence*, 2018.
- [28] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, “A model for types and levels of human interaction with automation,” *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans*, vol. 30, no. 3, pp. 286–297, 2000.
- [29] International Society of Automation, “Ansi/isa-95.00.01: Enterprise-control system integration,” Standard, 2010.
- [30] —, “ANSI/ISA-88.01:2010—batch control part 1: Models and terminology (international equivalent IEC 61512-1),” Standard, 2010.
- [31] International Electrotechnical Commission, “Iec 63278 (series): Asset administration shell for industrial applications,” International Standard, 2023.
- [32] X. Liu *et al.*, “AgentBench: Evaluating LLMs as agents,” in *Proc. ICLR*, 2024, arXiv preprint arXiv:2308.03688.
- [33] J. J. Downs and E. F. Vogel, “A plant-wide industrial process control problem,” *Computers & Chemical Engineering*, vol. 17, no. 3, pp. 245–255, 1993.
- [34] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur, “A dataset to support research in the design of secure water treatment systems,” in *Proc. 11th Int. Conf. Critical Information Infrastructures Security (CRITIS)*, 2017, pp. 88–99.
- [35] Q. Wu, G. Bansal, J. Zhang, T. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. Awadallah, R. White, D. Burger, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.

¹<https://github.com/kingdol666/AgentWorkShop> (platform and harness) and <https://github.com/kingdol666/plc-node-simulator> (PLC node simulator).

- [36] LangChain, Inc., “LangGraph,” [Online]. Available: <https://github.com/langchain-ai/langgraph>, 2024.
- [37] CrewAI, Inc., “CrewAI,” [Online]. Available: <https://github.com/crewAIInc/crewAI>, 2024.
- [38] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. ICLR*, 2023.
- [39] Anthropic, “Model context protocol,” [Online]. Available: <https://modelcontextprotocol.io>, 2024.
- [40] International Electrotechnical Commission, “IEC 62443: Security of industrial automation and control systems,” International Standard, 2018.
- [41] —, “IEC 61511: Functional safety—safety instrumented systems for the process industry sector,” International Standard, 2016.
- [42] Linux Foundation, “Agent2Agent (A2A) protocol,” [Online]. Available: <https://a2a-protocol.org>, 2025.
- [43] International Electrotechnical Commission, “IEC 62541: OPC unified architecture,” International Standard, 2020.
- [44] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in *Proc. ACM SIGIR*, 2009, pp. 758–759.
- [45] J. Carbonell and J. Goldstein, “The use of MMR, diversity-based reranking for reordering documents and producing summaries,” in *Proc. ACM SIGIR*, 1998, pp. 335–336.
- [46] Engineering Equipment and Materials Users Association, “Eemua 191: Alarm systems—a guide to design, management and procurement,” Guide, Edition 3, 2013.
- [47] Eclipse Foundation, “Eclipse ditto: Digital twin framework,” 2024.
- [48] International Organization for Standardization, “ISO 23247: Digital twin framework for manufacturing,” Standard, 2021.
- [49] L. Bainbridge, “Ironies of automation,” *Automatica*, vol. 19, no. 6, pp. 775–779, 1983.
- [50] J. D. Lee and K. A. See, “Trust in automation: Designing for appropriate reliance,” *Human Factors*, vol. 46, no. 1, pp. 50–80, 2004.
- [51] R. Parasuraman and D. H. Manzey, “Complacency and bias in human use of automation: An attentional integration,” *Human Factors*, vol. 52, no. 3, pp. 381–410, 2010.
- [52] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” *arXiv preprint arXiv:2302.12173*, 2023.
- [53] International Electrotechnical Commission, “IEC 62443-3-3: System security requirements and security levels,” Standard, 2023.
- [54] National Institute of Standards and Technology, “NIST SP 800-82 rev. 3: Guide to operational technology (OT) security,” Special Publication, 2023.